

ECE 310: Project 3 Report

Serial BCD Arithmetic Logic Unit

Ezana Enquobahrie

April 15, 2026

1 Introduction

This project is about designing and building an Arithmetic Logic Unit (ALU) that can perform addition and subtraction on 4-digit Binary Coded Decimal (BCD) numbers. Normally parallel ALUs need a lot of pins to route all the data, but this system is built for serial communication. By only using one input line and one output line, it saves a lot of hardware space, even though it takes more clock cycles to finish computing. The input data stream has a control sequence, an operation flag, and two 16-bit operands, all coming in through a single-bit serial line. The system does the math internally in parallel and then spits out a 5-digit (20-bit) BCD result serially, putting a hardcoded header (8'b10100101 or 8'hA5) at the front so downstream components know its coming.

2 Theory of Operation

2.1 BCD Addition and Hexadecimal Correction

Binary Coded Decimal uses 4 bits to represent standard decimal digits from 0 to 9 with an 8-4-2-1 weighted encoding. Even though 4 bits can technically represent 16 different values (0-15 in decimal or 0-F in hex), BCD only cares about the first 10. Because 6 of the possible 4-bit combinations are treated as invalid states we have to adjust standard binary addition to make it work right.

If a digits sum goes over 9 (for example, $6_{10} + 7_{10} = 13_{10}$, which is 1101_2 in raw binary), it ends up in the invalid hex space. To fix this and push the value into the proper BCD tens-column carry, 6_{10} (0110_2) is added to the raw result. This addition basically jumps over the 6 invalid states, fixing the 4-bit sum to 0011_2 (3) and correctly triggering a carry-out of 1 to the next digit to represent the 13.

2.2 BCD Subtraction via 10's Complement

BCD subtraction is a lot more complicated. In a normal binary ALU you can just do subtraction by taking the 2's complement of the subtrahend (flipping bits and adding 1). But since standard 4-bit BCD encoding isn't self-complementing in base-10, taking the 2's complement directly gives you mathematically wrong BCD results.

Instead, subtraction is done using the 10's complement method. You find the 9's complement of the subtrahend by subtracting each individual digit from 9. Then, 1_{10} is added to the least significant bit (I did this in hardware by setting the initial carry-in high) to get the exact 10's complement. This is great because it lets the system reuse the exact same addition hardware

for subtraction, saving a ton of logic space. For subtraction operations, if there is a final borrow (carry-out of the MSB) we just throw it away since the result is already formatted correctly.

3 System Architecture

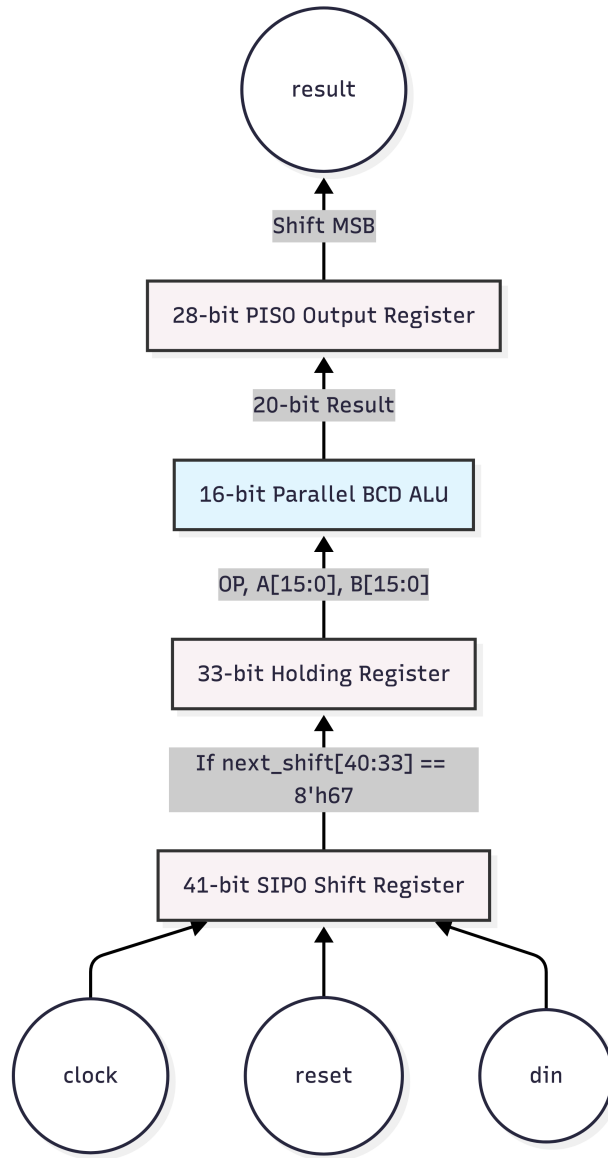


Figure 1: Top-level architecture of the Serial BCD ALU showing the continuous sequence integration between the SIPO, the cascaded ALUs, and the PISO register.

The system architecture is split up into three main functional blocks. The entire system is governed by a free-running clock and an active-high synchronous reset, ensuring all registers safely initialize to a known zero state on the next rising edge before any operations begin. This abstraction keeps the clock-dependent serial shifting logic completely separate from the purely combinational parallel math logic.

3.1 Serial-In Parallel-Out (SIPO) and Sequence Detection

The input stream constantly shifts into a 41-bit register on every positive clock edge. The system is always watching the top 8 bits for the control pattern `8'h67`, which acts like a sync packet. Once it spots it, the 33-bit payload (the 1-bit operation flag and the two 16-bit operands) gets locked into a parallel holding register immediately so the combinational ALU can start calculating without having to wait for the serial line.

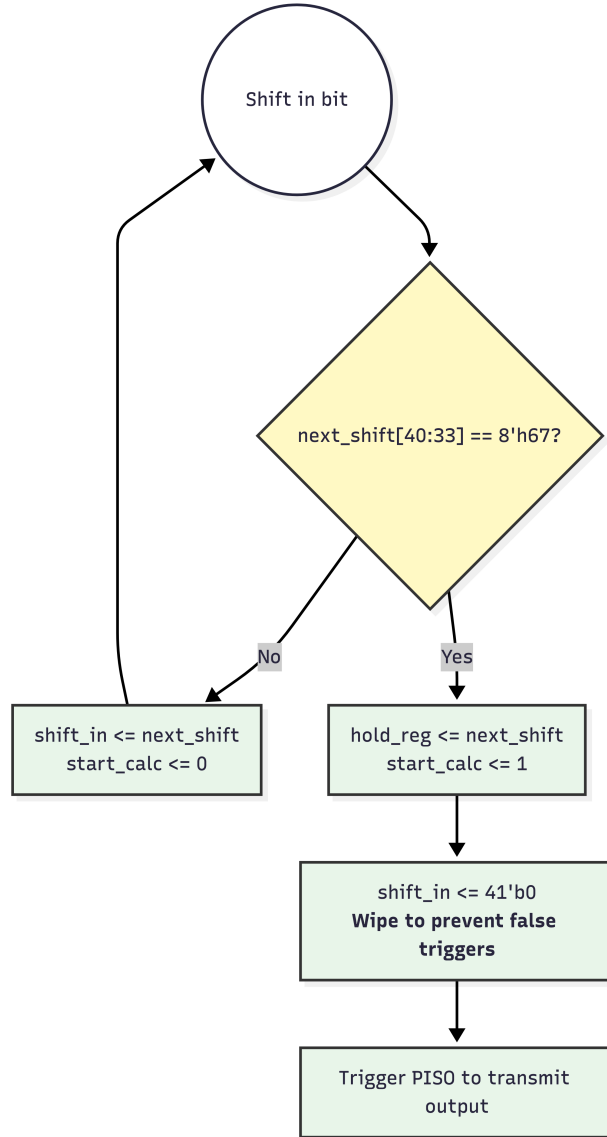


Figure 2: Sequence detection flowchart handling continuous bit-stream inputs and shift-register clearing.

3.2 The 16-bit BCD ALU and Carry Routing

The arithmetic core is purely combinational, basically just chaining four 4-bit BCD ALUs together. This lets the 16-bit calculation ripple through as fast as the silicon path allows, totally independent of the main system clock.

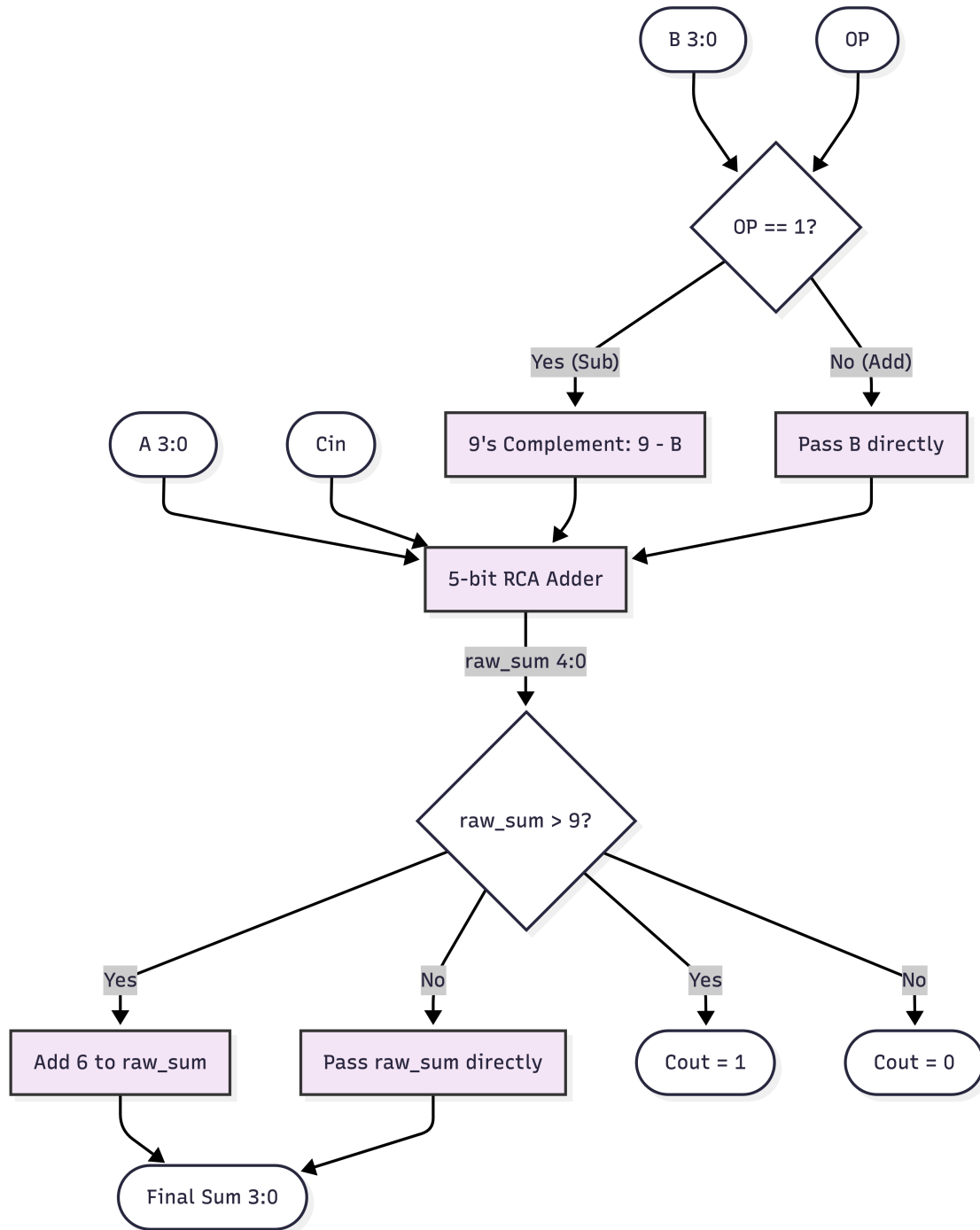


Figure 3: Internal datapath of a single 4-bit BCD digit calculator showing the 9's complement MUX, the 5-bit raw adder, and the +6 adjustment combinational block.

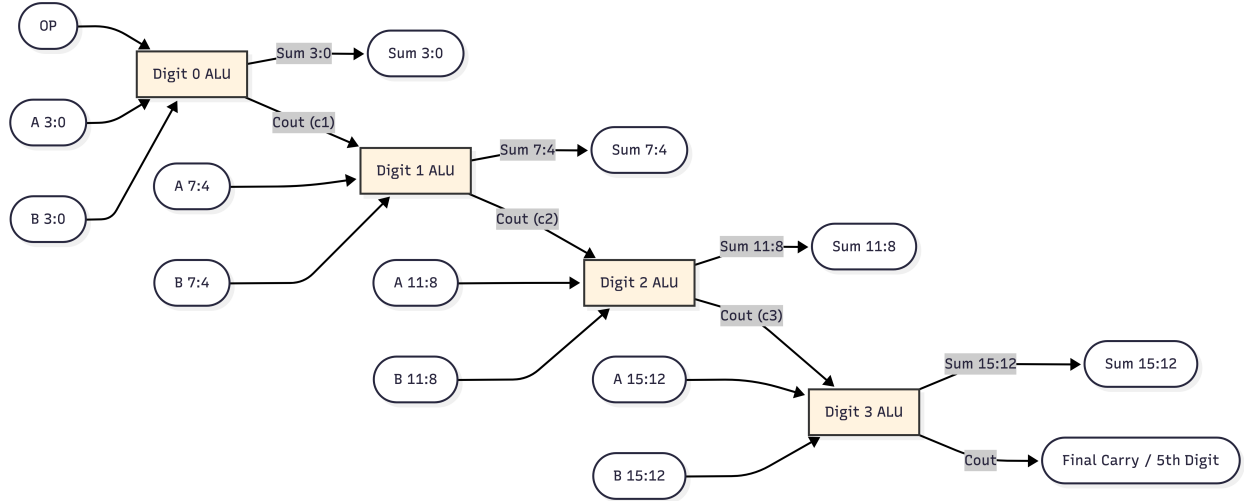


Figure 4: Cascaded 16-bit parallel ALU setup detailing the ripple-carry routing between digits.

4 Verilog Implementation

4.1 Sequence Detection and Embedded Header Prevention

One of the hardest parts of this project was dealing with continuous inputs where the data payload itself might accidentally contain the 8'h67 header sequence (an "embedded header"). If the system wasn't designed carefully, random data shifting through the register could trigger a fake synchronization event.

To stop this false sequence detection from happening, the hardware was set up so that the `shift_in` register gets completely wiped to zero the exact moment a valid header is verified.

```

1 // Check for control header 8'h67
2 if (next_shift[40:33] == 8'h67) begin
3     hold_reg <= next_shift[32:0];
4     start_calc <= 1'b1;
5
6     // Clear the shift register entirely. This prevents payload data
7     // from continuously shifting and falsely triggering the header
8     // detector on subsequent clock cycles.
9     shift_in <= 41'b0;
10 end else begin
11     shift_in <= next_shift;
12     start_calc <= 1'b0;
13 end

```

Listing 1: Sequence Detection and Payload Isolation logic.

4.2 Single Digit ALU Logic

Each individual digit ALU uses a multiplexer to smoothly switch between adding and subtracting. By routing the B input through a 9's complement subtractor when the OP flag is high, the standard 5-bit adder can be reused for both mathematical operations.

```

1 // 9's complement of B for subtraction
2 assign B_mod = OP ? (4'd9 - B) : B;

```

```

3
4 // 5-bit addition (Cin handles the +1 for 10's complement)
5 assign raw_sum = A + B_mod + Cin;
6
7 // Check for invalid BCD range (>9)
8 assign adjust = (raw_sum > 9);
9
10 // BCD adjustment: add 6 if > 9 and trigger carry out
11 assign Sum = adjust ? (raw_sum + 3'd6) : raw_sum;
12 assign Cout = adjust;

```

Listing 2: 4-bit ALU Core Logic handling 10's complement and BCD adjustment.

5 Simulation and Verification

5.1 Test Vector Justification

To make sure all the operational constraints were met before synthesis, I picked specific test vectors to verify standard math, edge-case carries, and to make sure the sequence detector was robust against tricky inputs.

Table 1: Summary of test vectors used for behavioral simulation and verification.

A	B	OP	Expected Result	Verification Goal
16'h3627	16'h1287	0 (Add)	20'h04914	Verify basic cascaded addition with standard BCD carry adjustments.
16'h0637	16'h0459	1 (Sub)	20'h00178	Verify 10's complement subtraction and proper borrow discard.
16'h6767	16'h6767	0 (Add)	20'h13534	Verify "Embedded Header" rejection. Tests if the system falsely triggers on data matching 8'h67.

5.2 Simulation Waveforms

A strong Verilog testbench was used to inject serial packets bit-by-bit into the module to check timing accuracy and combinational stability.

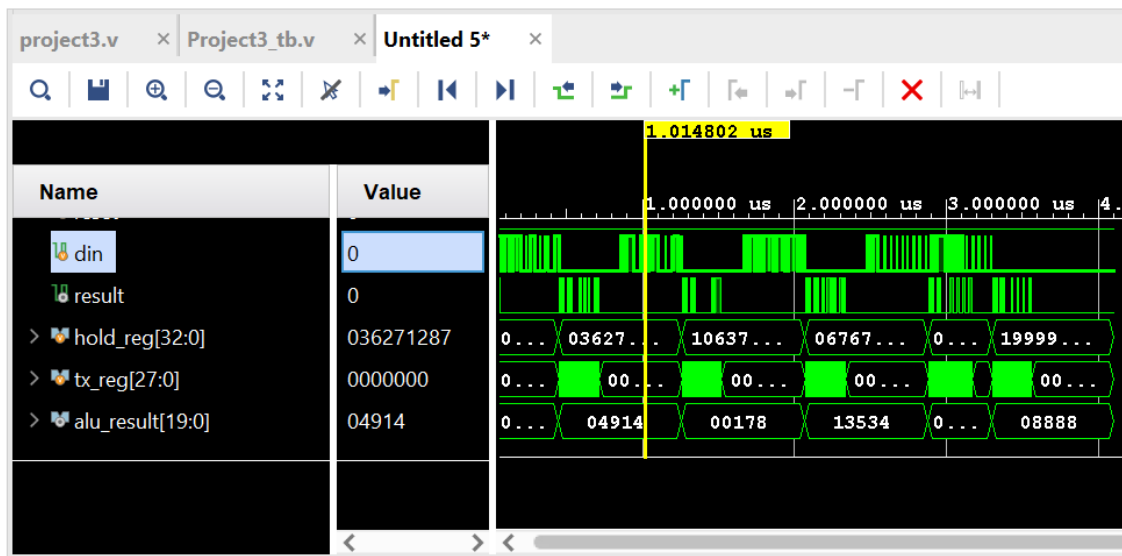


Figure 5: Timing waveform demonstrating successful serial addition of $3627 + 1287 = 04914$ and proper PISO transmission.

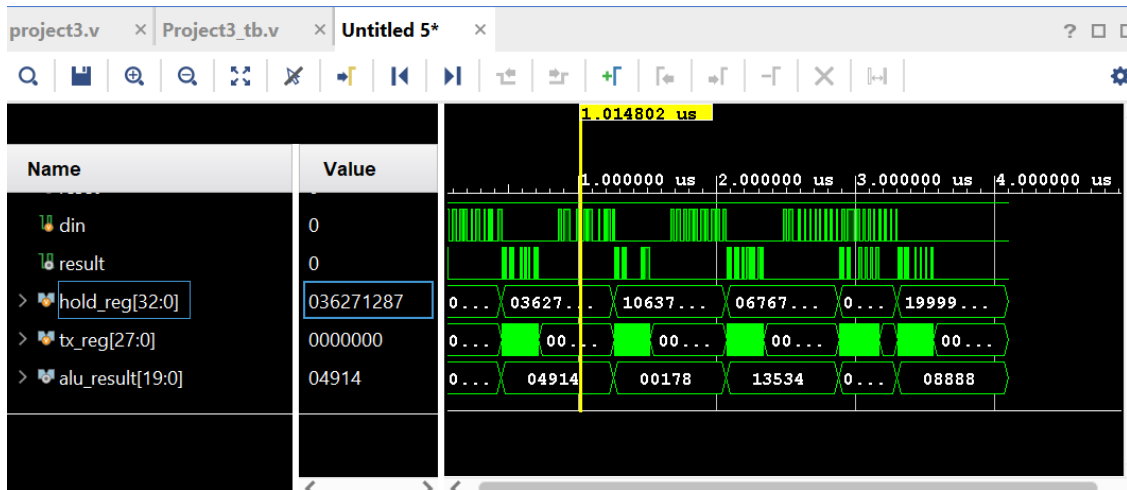


Figure 6: Timing waveform demonstrating successful serial subtraction of $637 - 459 = 00178$ and borrow elimination.

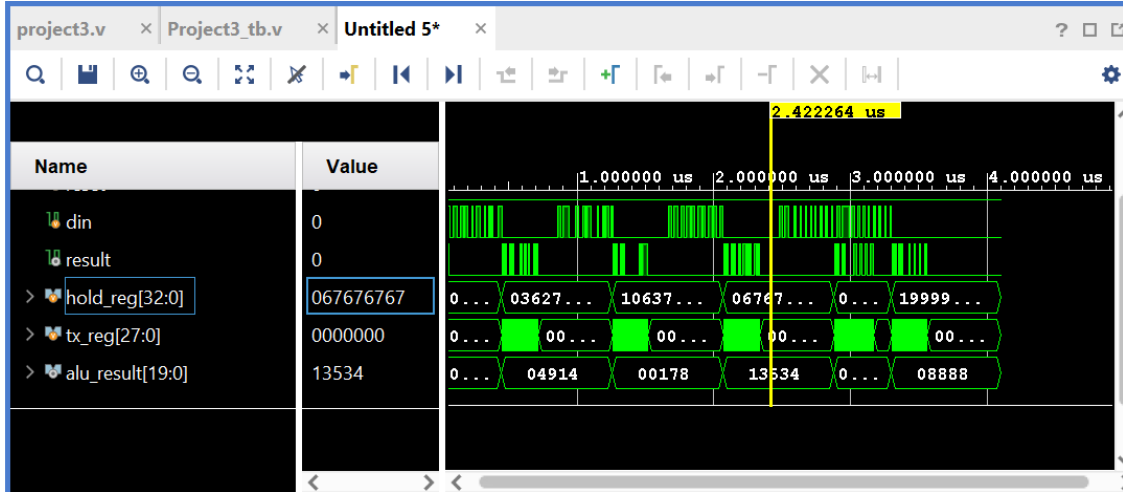


Figure 7: Timing waveform showing two back-to-back packets being processed proving uninterrupted continuous input capabilities.

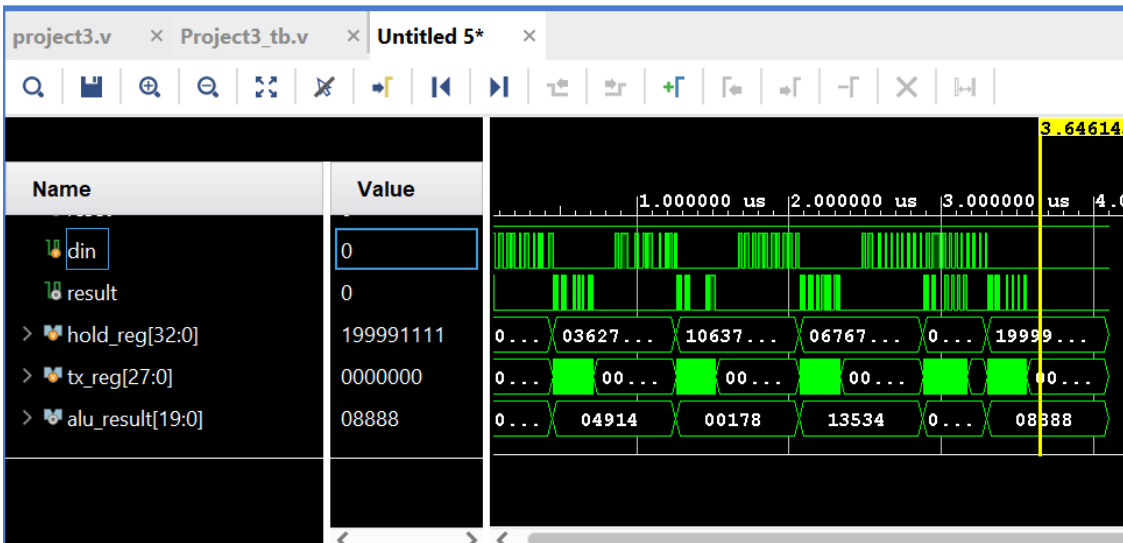


Figure 8: Timing waveform verifying the sequence detector correctly ignores embedded 8'h67 sequences within the operand payload.

6 RTL Synthesis

The behavioral code was run through Vivado's elaboration engine to verify that the synthesizable subset of Verilog actually mapped to real logical primitives.

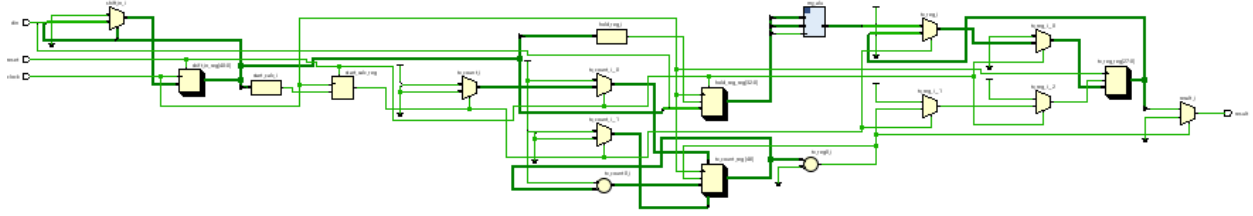


Figure 9: Vivado RTL Elaborated Design schematic showing the Project3 top module inferring the SIPO register, the instantiated BCD ALU block, and the PISO.

7 Conclusion

Overall, the Serial BCD ALU was successfully designed, built, and verified using Verilog. Making the design modular separated the 16-bit parallel math logic from the serial interface, which reduced routing complexity a lot. By clearing the input register right after detecting a header, the sequence detector handled embedded header bugs perfectly without messing up the math datapath. The simulation waveforms and Gradescope autograder results showed it had 100% functional accuracy across all the test parameters, including continuous asynchronous inputs and proper 10's complement adjustments.